VOLUME 12 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 12

ADVANCED JAVA, TESTING & PRODUCTION

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 12

1. About This Module

Welcome to Volume 12 of the IT Study Hub Complete Java Programming Series. This is the grand finale. In previous modules, you learned the language, database connections, and the Spring Boot framework. You can build an application—but can you guarantee it won't crash when millions of users access it? Can you prove to your boss that it works without manually testing it? Can you deploy it securely to a cloud server?

This final module transforms you from a "coder" into a **Professional Software Engineer**. You will master the final critical steps of the software lifecycle: Architecture, Automated Testing, Security, Containerization (Docker), and Continuous Integration/Deployment (CI/CD).

2. Learning Outcomes

- Architect enterprise apps using **Design Patterns** and **SOLID Principles**.
- Write unbreakable code using automated **Unit Tests (JUnit)** and **Mocking (Mockito)**.
- Monitor your applications using professional **Logging (SLF4J)** and document APIs with **Swagger**.
- Secure your applications using **JWT (JSON Web Tokens)** and OAuth2 concepts.
- Understand Java performance, the JVM, and Garbage Collection tuning.
- Package applications into portable environments using **Docker**.
- Automate the deployment pipeline using **Git** and **CI/CD** basics.

3. Module Coverage

We start by exploring Clean Architecture and Design Patterns. Then we tackle Test-Driven Development (TDD) with JUnit and Mockito. We move into production monitoring with Logging and Swagger. Next, we secure our endpoints with JWTs. Finally, we step out of Java into the DevOps world, exploring Docker, Git, and CI/CD pipelines, culminating in a massive "Production-Ready Java REST API" project and the Java Developer Roadmap.

IT STUDY HUB

4. Advanced Java Architecture & SOLID Principles

Writing code that "works" is easy. Writing code that is easy to read, modify, and extend 5 years from now is extremely difficult. **Clean Architecture** and the **SOLID principles** are the rules professionals follow to prevent "spaghetti code".

Clean Architecture vs MVC

Traditional MVC	Clean Architecture
User → Controller → Service → Database *(Everything depends on the Database. Hard to swap DB out.)*	Core Domain (Inner Circle) ← Controllers/DB (Outer Circle) *(The core logic depends on NOTHING. DB and Web UI are just plugins.)*

IT STUDY HUB

The SOLID Principles

Principle	Explanation	Why it matters
Single Responsibility	A class should have one, and only one, reason to change.	If a class handles both Database logic and Email logic, changing Email logic might accidentally break the Database code.
Open-Closed	Classes should be open for extension, but closed for modification.	You should be able to add new features by adding new classes, without editing old, already-tested code.
Liskov Substitution	Child classes must be perfectly substitutable for their parent classes.	If a 'Bird' parent class has a 'fly()' method, a 'Penguin' child class breaks this principle because penguins can't fly.
Interface Segregation	Many client-specific interfaces are better than one general-purpose interface.	Don't force a 'Smartwatch' class to implement a 'Printable' interface just because they share a generic 'Machine' parent.
Dependency Inversion	High-level modules should not depend on low-level modules. Both should depend on abstractions.	This is why we inject Interfaces (like `UserRepository`) into Services, rather than exact MySQL implementations.

5. Design Patterns

Design patterns are standardized, proven solutions to common software design problems. They are not code you copy-paste, but rather structural blueprints for solving specific issues.

IT STUDY HUB

Concept: Singleton Pattern

Ensures a class has only ONE instance globally, and provides a global point of access to it.

> **Simple Explanation:**
> You make the constructor `private` so nobody can use `new`. You create a `public static` method that returns the sole instance. If the instance doesn't exist, it creates it. If it does, it returns the existing one.

> ***Real-Life Analogy:***
> *A country can have only one President. If citizen A asks to speak to the President, and citizen B asks to speak to the President, they both get directed to the exact same person.*

> **Why It Matters:**
> Prevents system exhaustion. You only need ONE database connection pool manager for an entire application. Creating multiple would crash the database.

Syntax / Structure:

```
public class Singleton {
    private static Singleton instance;
    private Singleton() {} // Private!
    public static Singleton getInstance() {
        if(instance == null) instance = new Singleton();
        return instance;
    }
}
```

Example Code:

```
public class DatabaseConfig {
    // 1. Static variable holding the only instance
    private static DatabaseConfig instance;

    public String dbUrl;

    // 2. Private constructor prevents 'new DatabaseConfig()'
    private DatabaseConfig() {
```

```
        this.dbUrl = "jdbc:postgresql://localhost:5432/prod_db";
    }

    // 3. Global access point
    public static DatabaseConfig getInstance() {
        if (instance == null) {
            instance = new DatabaseConfig();
        }
        return instance;
    }
}

public class SingletonTest {
    public static void main(String[] args) {
        DatabaseConfig config1 = DatabaseConfig.getInstance();
        DatabaseConfig config2 = DatabaseConfig.getInstance();

        System.out.println("Are they the exact same object in memory? " + (config1 == co
    }
}
```

```
Expected Output:
Are they the exact same object in memory? true
```

Code Explanation:
Both `config1` and `config2` point to the exact same memory address. The private constructor effectively banned the `new` keyword from being used outside the class.

> ***Real-world Use:***
> *Spring Boot uses Singletons heavily. By default, every `@Service` and `@Repository` you create is a Singleton managed by the Spring IoC container.*

> **Common Mistakes:**
> - Making a Singleton in a multithreaded application without using the `synchronized` keyword on `getInstance()`. Two threads might hit `if(instance == null)` simultaneously and create TWO objects!

Best Practices:

- In modern Java, Enums are the safest way to implement Singletons. Or, just let Spring Framework handle it via `@Component`.

IT STUDY HUB

Concept: Factory Pattern

Creates objects without exposing the instantiation logic to the client.

> **Simple Explanation:**
> Instead of writing `new Car()`, `new Truck()`, or `new Bike()` throughout your code, you ask a `VehicleFactory` to build it for you by passing a string like 'CAR'.

> ***Real-Life Analogy:***
> *Ordering food at a restaurant. You don't go to the kitchen, gather ingredients, and build a Pizza (using `new`). You ask the Waiter (Factory) for a 'Pizza', and they hand you the finished product.*

> **Why It Matters:**
> If the process of building a Car changes in the future, you only update the Factory class, rather than hunting down every `new Car()` in your entire codebase.

Example Code:

```
interface Notification {
    void notifyUser();
}

class EmailNotification implements Notification {
    public void notifyUser() { System.out.println("Sending an Email..."); }
}

class SMSNotification implements Notification {
    public void notifyUser() { System.out.println("Sending an SMS..."); }
}

// THE FACTORY
class NotificationFactory {
    public Notification createNotification(String type) {
        if (type.equals("SMS")) return new SMSNotification();
        else if (type.equals("EMAIL")) return new EmailNotification();
        else throw new IllegalArgumentException("Unknown channel");
    }
}
```

```
public class FactoryDemo {
    public static void main(String[] args) {
        NotificationFactory factory = new NotificationFactory();

        // The client doesn't know HOW the object is built, just asks for it!
        Notification n1 = factory.createNotification("EMAIL");
        n1.notifyUser();
    }
}
```

```
Expected Output:
Sending an Email...
```

Code Explanation:
The `FactoryDemo` class doesn't have any hardcoded link to `EmailNotification`. It only knows about the `Notification` interface and the Factory. This perfectly follows the Dependency Inversion SOLID principle.

> ***Real-world Use:***
> *Generating different types of Database connections (MySQL vs Oracle) based on a configuration file string.*

> **Common Mistakes:**
>
> - Using a Factory when object creation is incredibly simple. It adds unnecessary classes and complexity (over-engineering).

> **Best Practices:**
>
> - Use the Factory pattern when the creation of an object involves complex logic, or when you need to switch implementations at runtime.

---

6. Automated Testing (JUnit & Mockito)

In production, you cannot test a banking app by manually clicking buttons to see if money transfers correctly. Automated Testing involves writing code that tests your code. If you change something later, you run your tests; if they pass, you know you didn't break anything.

Unit Testing vs Integration Testing

Type	Definition	Speed & Scope
Unit Testing (JUnit/ Mockito)	Testing a single class or method in complete isolation.	Extremely fast (milliseconds). Fakes (Mocks) the database.
Integration Testing	Testing how multiple layers (Controller + DB) work together.	Slower. Uses a real, temporary database (like H2).

IT STUDY HUB

Concept: Unit Testing with JUnit

A framework for writing assertions that verify your method outputs match expectations.

> **Simple Explanation:**
> You provide a method with test inputs. You run the method. You `assert` (demand) that the output equals a specific value. If it matches, the test passes (Green). If it doesn't, the test fails (Red).

> ***Real-Life Analogy:***
> *A teacher grading a math test. The test asks '2 + 2'. The student's code answers '4'. The teacher looks at the answer key (Assertion) and marks it correct.*

> **Why It Matters:**
> Without unit tests, updating a legacy codebase is terrifying because you never know what you might accidentally break. Tests provide a safety net.

Syntax / Structure:

```
@Test
void testAddition() {
    int result = math.add(2, 2);
    assertEquals(4, result);
}
```

Example Code:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class MathUtils {
    public int multiply(int a, int b) { return a * b; }
    public int divide(int a, int b) { return a / b; }
}

public class MathTest {

    @Test
```

```
    void testMultiplication() {
        MathUtils calc = new MathUtils();
        int result = calc.multiply(5, 4);

        // Assert that we EXPECT 20, and ACTUAL is result
        assertEquals(20, result, "5 * 4 should be 20");
    }

    @Test
    void testDivisionByZero() {
        MathUtils calc = new MathUtils();

        // Assert that dividing by zero throws an ArithmeticException!
        assertThrows(ArithmeticException.class, () -> {
            calc.divide(10, 0);
        });
    }
}
```

```
Expected Output:
[Test Results: 2/2 Tests Passed - GREEN]
```

Code Explanation:
The `@Test` annotation tells JUnit to run these methods during the build phase. `assertEquals` checks exact values. `assertThrows` uses a Lambda to execute risky code and checks if the expected exception is correctly thrown.

> ***Real-world Use:***
> *Every single Enterprise Java project uses JUnit. Builds in a CI/CD pipeline will automatically abort if even one JUnit test fails.*

> **Common Mistakes:**
>
> - Writing tests that depend on a real database or internet connection. If the internet drops, your test fails, even if your code is perfect. Unit tests must be 100% isolated.

> **Best Practices:**
>
> - Follow the AAA pattern in testing: Arrange (setup data), Act (call the method), Assert (check the result).

IT STUDY HUB

Concept: Mocking with Mockito

Creating fake, programmable versions of external dependencies so you can test a single class in isolation.

Simple Explanation:
If your `UserService` needs to talk to the `UserRepository` (Database), you can't unit test the Service without a database. Mockito lets you create a 'Mock' (fake) Repository. You program the fake repository to say: 'If anyone asks you for User 1, instantly hand them this fake User object.'

Real-Life Analogy:
Running a fire drill. You don't actually light the building on fire (the real Database). You use a smoke machine (the Mock) to simulate the fire, so you can test if the employees (your Service logic) know how to evacuate safely.

Why It Matters:
It makes unit tests infinitely faster and removes network/database flakiness.

Syntax / Structure:

```
@Mock
UserRepository repo;

@InjectMocks
UserService service;

// Train the mock!
when(repo.findById(1)).thenReturn(fakeUser);
```

Example Code:

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;

// Interface simulating Database interaction
interface UserRepository {
    String getUserName(int id);
}

// The class we ACTUALLY want to test
class UserService {
    private UserRepository repo;
    public UserService(UserRepository repo) { this.repo = repo; }

    public String getGreeting(int id) {
        String name = repo.getUserName(id);
        return "Welcome, " + name + "!";
    }
}

// The Test Class
@ExtendWith(MockitoExtension.class)
public class UserServiceTest {

    @Mock // Creates a fake database connection!
    UserRepository fakeRepo;

    @InjectMocks // Injects the fake database into the Service
    UserService service;

    @Test
    void testGreeting() {
        // 1. ARRANGE: Train the mock!
        Mockito.when(fakeRepo.getUserName(101)).thenReturn("Alice");

        // 2. ACT: Call the real service
        String result = service.getGreeting(101);

        // 3. ASSERT: Verify the result
        assertEquals("Welcome, Alice!", result);
    }
}
```

```
Expected Output:
[Test Results: 1/1 Tests Passed - GREEN]
```

Code Explanation:
We didn't need a real MySQL database! `@Mock` created a dummy `UserRepository`. We used `Mockito.when()` to train it: 'If anyone asks for ID 101, return Alice'. The `UserService` didn't know it was talking to a fake database. It processed the 'Alice' string perfectly.

> ***Real-world Use:***
> *Mocking Stripe or PayPal payment gateways. You don't want to actually charge credit cards every time you run your automated test suite!*

> **Common Mistakes:**
>
> - Mocking the class you are actually trying to test. You only mock the *dependencies* (the things the class talks to).

> **Best Practices:**
>
> - Use `@ExtendWith(MockitoExtension.class)` in JUnit 5 to automatically initialize all `@Mock` and `@InjectMocks` annotations cleanly.

---

7. Logging and API Documentation

IT STUDY HUB

Concept: Logging (SLF4J and Logback)

Recording application events to a file or console to track errors and user activity in production.

Simple Explanation:
`System.out.println()` is banned in professional software. It is slow, cannot be turned off easily, and cannot be saved to a rolling file. We use a Logger. Loggers have 'levels' (INFO, WARN, ERROR). In production, you can tell the server 'Only print ERRORs, ignore the rest' to save hard drive space.

Real-Life Analogy:
A ship's captain logbook. They don't write down every breath they take (Debug level). They write down when they leave port (Info level) and when they hit an iceberg (Error level).

Why It Matters:
When a server crashes at 3:00 AM, the log file is the only evidence you have to figure out what went wrong.

Syntax / Structure:

```
private static final Logger log = LoggerFactory.getLogger(MyClass.class);
log.info("User {} logged in", userId);
log.error("Database connection failed", exception);
```

Code Explanation:
By using `{}` placeholders, SLF4J efficiently injects variables into the string. If the logging level is turned off, it skips the string processing entirely, saving CPU cycles.

Real-world Use:
Every enterprise app writes logs to files (like `app.log`). Tools like Splunk or ELK (Elasticsearch, Logstash, Kibana) scrape these files and create visual dashboards of server health.

Common Mistakes:

- Logging sensitive data (like passwords, credit card numbers, or full JWT tokens) in plain text log files. This is a severe security breach.

Best Practices:

- Use `log.debug()` for development variables, `log.info()` for significant state changes (user login), and `log.error()` strictly inside `catch` blocks.

REST API Documentation (Swagger / OpenAPI)

When you build a REST API, frontend developers (React/Android) need to know what URLs to call, what JSON to send, and what JSON to expect back. Writing this manually in a Word document is terrible because the code changes constantly.

Swagger (OpenAPI) automatically reads your Spring Boot `@RestController` annotations and generates a beautiful, interactive website where developers can see your endpoints and even test them directly from the browser.

```
GET  /api/students

POST /api/students
     body: { "name": "string", "email": "string" }
```

Adding `springdoc-openapi-starter-webmvc-ui` to your Maven pom.xml generates this UI automatically at `http://localhost:8080/swagger-ui.html`.

---

8. Security: Authentication & Authorization (JWT)

Authentication vs Authorization

Concept	Definition	Real-Life Analogy
Authentication (AuthN)	Verifying *Who you are*.	Checking your ID at the airport security gate.
Authorization (AuthZ)	Verifying *What you can do*.	The flight attendant checking if your ticket is for Economy or First Class.

IT STUDY HUB

Concept: JSON Web Tokens (JWT)

A stateless, cryptographically signed token used to secure REST APIs.

> **Simple Explanation:**
> REST APIs are 'Stateless'—they have no memory. They don't remember if you logged in 5 seconds ago. A JWT is a digital VIP pass. When you log in, the server signs a JWT and hands it to you. For every future request, you show the JWT. Because it is cryptographically signed, the server trusts it without checking the database again.

> ***Real-Life Analogy:***
> *A passport stamped by a government. The border agent (Server) doesn't need to call the government database every time you cross; they just look at the cryptographic stamp (Signature) on the passport (JWT) to verify it's legitimate.*

> **Why It Matters:**
> It makes backend servers extremely fast and scalable because they don't have to save user 'Sessions' in RAM.

Example Code:

```
// Conceptual JWT Payload Structure (Decoded)
{
  "sub": "alice@studyhub.com", // Subject (Who it belongs to)
  "roles": ["ROLE_ADMIN"],     // Authorization data
  "iat": 1691234567,           // Issued At (Timestamp)
  "exp": 1691238167            // Expiration (1 hour later)
}

// ------------------------------------------------
// Spring Security Conceptual Workflow:
// 1. Client sends POST /login with { "username":"alice", "password":"123" }
// 2. Server verifies DB. Generates JWT String: "eyJhbGciOiJIUzI1... (Encoded)"
// 3. Client sends GET /api/data with Header: "Authorization: Bearer eyJhbGci..."
// 4. Spring Security validates the Signature using its Secret Key. Allows access.
```

Code Explanation:
A JWT has three parts: Header, Payload (the JSON above), and Signature. The Payload is

Base64 encoded (NOT encrypted), meaning anyone can read it. However, if a hacker changes their role from 'USER' to 'ADMIN', the Signature breaks, and the Server instantly rejects it.

Real-world Use:
Used by virtually all modern Single Page Applications (React/Angular) and Mobile Apps to communicate securely with Java backends.

Common Mistakes:

- Putting sensitive data (like passwords or credit card numbers) inside the JWT payload. Anyone who intercepts the token can decode and read the payload!

Best Practices:

- Keep JWT lifespans extremely short (e.g., 15 minutes) and use 'Refresh Tokens' to get new ones. This minimizes damage if a token is stolen by a hacker.

IT STUDY HUB

9. Performance, JVM, and Garbage Collection

Professional Java developers must understand how the Java Virtual Machine (JVM) manages memory to prevent applications from crashing with `OutOfMemoryError`.

JVM Memory Structure

Stack Memory

Fast, small, auto-managed.

Stores local primitive variables (int, double) and method calls.

Each Thread has its own Stack.

Heap Memory

Large, globally shared.

Stores ALL Objects (e.g., `new Student()`, Arrays).

Managed by Garbage Collection.

IT STUDY

Concept: Garbage Collection (GC)

An automatic process inside the JVM that finds and destroys objects in Heap memory that are no longer being used.

> **Simple Explanation:**
> In older languages like C++, you had to manually delete objects from memory. If you forgot, the computer ran out of RAM (Memory Leak). In Java, the Garbage Collector acts like a janitor. It scans the Heap. If it finds an Object that no variable is pointing to anymore, it sweeps it into the trash, freeing up RAM.

> ***Real-Life Analogy:***
> *A table at a restaurant. If a customer is sitting there (Reference), the table is in use. Once the customer leaves and the table is empty (Unreferenced Object), the busboy (Garbage Collector) cleans the table so a new customer can use it.*

> **Why It Matters:**
> If your code holds onto objects unnecessarily (like keeping old users in a massive static List), the GC cannot delete them. This causes a Memory Leak, eventually crashing your server.

Syntax / Structure:

```
System.gc(); // Suggests a GC run, but JVM usually ignores this.
```

Code Explanation:
You almost never interact with the GC in code. Instead, you tune it via JVM startup flags. E.g., `java -Xmx2G -jar app.jar` tells the JVM to allocate exactly 2 Gigabytes of RAM for the Heap.

> ***Real-world Use:***
> *Enterprise apps use tools like VisualVM or Datadog to monitor JVM Heap usage in real-time. If they see a continuous diagonal line going up, they know a memory leak is occurring.*

Common Mistakes:

- Calling `System.gc()` manually. It forces the JVM to pause all processing (Stop-The-World event) to clean memory, causing massive lag spikes in production servers.

Best Practices:

- Ensure you close all resources (Files, DB Connections) using try-with-resources. Open resources block the GC from doing its job.

IT STUDY HUB

10. DevOps: Docker, Git, and CI/CD

Writing code is only half the job. **DevOps** is the process of getting that code off your laptop, packaging it safely, and deploying it to a live cloud server (like AWS or Azure) for users to access.

IT STUDY HUB

Concept: Docker Fundamentals

A platform that packages an application and all its dependencies into a single, standardized unit called a Container.

Simple Explanation:
The classic developer excuse is 'It works on my machine!' because their laptop has a specific version of Java or MySQL. Docker puts your Java App, the specific Java version, and the OS environment into a virtual 'Shipping Container'. This container will run exactly the same way on any computer on Earth.

Real-Life Analogy:
Moving a fish. Instead of just handing you the fish (the .jar file) and hoping you have the right water temperature, I hand you a sealed aquarium with the fish, the exact water, and the exact temperature inside (the Docker Container).

Why It Matters:
It makes cloud deployment infinitely scalable. AWS doesn't need to know anything about Java; it just needs to know how to run a Docker Container.

Syntax / Structure:

```
# Sample Dockerfile for a Spring Boot App
FROM openjdk:17-jdk-slim
COPY target/myapp.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Code Explanation:
This `Dockerfile` is a recipe. Line 1: Start with a clean Linux environment with Java 17 pre-installed. Line 2: Copy our compiled Spring Boot `myapp.jar` file into the container. Line 3: The command to run when the container boots up.

Real-world Use:
Netflix runs entirely on hundreds of thousands of Docker containers spun up and destroyed dynamically based on viewer traffic.

Common Mistakes:

- Putting passwords or database credentials hardcoded inside a Dockerfile. Anyone who downloads the container can extract them.

Best Practices:

- Use Environment Variables to pass secrets into a Docker container at runtime.

CI/CD Basics (Continuous Integration / Continuous Deployment)

CI/CD is an automated pipeline. When you push your code to GitHub, the pipeline automatically wakes up, builds the code, runs your JUnit tests, builds a Docker image, and deploys it to the live server. Humans do not deploy code manually in the modern era.

Stage	Action	Tool Used
Code	Developer writes code and pushes to repo.	Git / GitHub
Build & Test (CI)	Compiles code, runs JUnit/Mockito tests. Aborts if a test fails!	GitHub Actions / Jenkins / Maven
Package	Creates the Docker Container image.	Docker
Deploy (CD)	Pushes the container to the live cloud server.	AWS / Kubernetes / Azure

---

11. FINAL PROJECT: Production-Ready Java REST API

This is the synthesis of the entire 12-volume series. We are going to architect a secure, scalable, tested, and deployable Spring Boot application. Due to the scale of a production app, we will display the critical architectural components.

1. Clean Project Architecture

```
src/main/java/com/itstudyhub/production/
 ├── config/            (Swagger & Security Configuration)
 ├── controller/        (REST Endpoints - Web Layer)
 ├── service/           (Business Logic & Interfaces)
 ├── repository/        (Spring Data JPA - Database Layer)
 ├── model/             (JPA Entities like 'User' and 'Product')
 ├── dto/               (Sanitized Records for Requests/Responses)
 ├── exception/         (GlobalExceptionHandler & Custom Exceptions)
 └── security/          (JWT Utility Classes)
src/test/java/com/itstudyhub/production/
 ├── controller/        (Integration Tests)
 └── service/           (Unit Tests with Mockito)
Dockerfile              (For packaging)
.github/workflows/      (CI/CD Pipeline YAML)
```

2. The Controller (Web Layer) with Logging & DTOs

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/v1/products")
public class ProductController {

    // Professional SLF4J Logger
```

```
    private static final Logger log = LoggerFactory.getLogger(ProductController.class);

    private final ProductService productService; // Dependency Injection

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @PostMapping
    public ResponseEntity createProduct(@Valid @RequestBody ProductRequestDTO dto) {
        log.info("Received request to create product: {}", dto.name());

        ProductResponseDTO response = productService.createProduct(dto);

        log.info("Successfully created product ID: {}", response.id());
        return ResponseEntity.status(201).body(response); // HTTP 201 Created
    }
}
```

3. The Service (Business Logic) & Custom Exceptions

```
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    public ProductResponseDTO createProduct(ProductRequestDTO dto) {
        // Business Rule: Ensure product name is unique
        if (repository.existsByName(dto.name())) {
            // Throw custom exception to be caught by GlobalExceptionHandler
            throw new DuplicateResourceException("Product name already exists!");
        }

        // Map DTO to Entity
```

```
        Product entity = new Product();
        entity.setName(dto.name());
        entity.setPrice(dto.price());

        // Save to Database
        Product savedEntity = repository.save(entity);

        // Map Entity back to DTO for safe response
        return new ProductResponseDTO(savedEntity.getId(), savedEntity.getName());
    }
}
```

4. Unit Testing the Service (JUnit 5 + Mockito)

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    ProductRepository mockRepo; // Fake Database!

    @InjectMocks
    ProductService productService; // Service using the fake database

    @Test
    void testCreateProduct_ThrowsException_WhenDuplicateName() {
        // ARRANGE: Train the mock database to return TRUE for duplicate check
        ProductRequestDTO request = new ProductRequestDTO("Laptop", 1000.0);
        Mockito.when(mockRepo.existsByName("Laptop")).thenReturn(true);

        // ACT & ASSERT: Ensure the Service throws our exact custom exception
        assertThrows(DuplicateResourceException.class, () -> {
            productService.createProduct(request);
```

```
        });
    }
}
```

Project Synthesis Explanation:

1. **Security & Safety:** We used `ProductRequestDTO` to safely accept JSON from the user, and `ProductResponseDTO` to safely send data back. The database Entity (`Product`) is never directly exposed to the internet.
2. **Professional Logging:** `log.info()` is used instead of `System.out.println()`, embedding timestamps and thread IDs into a server log file automatically.
3. **Test-Driven Reliability:** By using `@Mock`, we tested the `ProductService` logic perfectly without ever turning on MySQL or booting up Spring Boot. The test runs in milliseconds. In a CI/CD pipeline, if a developer breaks the duplicate name logic, this test will fail, and the pipeline will block the deployment to the live server.

IT STUDY HUB

12. Production Deployment Checklist

Pre-Launch Verification

- **Security:** Are all passwords/API Keys moved to external Environment Variables? (Never commit them to Git).
- **Security:** Are JWT secrets sufficiently long (e.g., 256+ bit HMAC)?
- **Database:** Are you using a proper SQL database (MySQL/PostgreSQL) instead of in-memory H2?
- **Performance:** Is Connection Pooling (HikariCP) properly configured?
- **Testing:** Do all Unit and Integration tests pass in the CI/CD pipeline?
- **Monitoring:** Is SLF4J logging properly rolling over daily into a `.log` file?
- **Documentation:** Is the Swagger/OpenAPI UI enabled for frontend developers to reference?
- **Containerization:** Does the Dockerfile successfully build and run the `.jar` file?

IT STU

13. Module Assessment and Interview Prep

Test your mastery of Enterprise Architecture, Testing, Security, and DevOps.

A. Practice Questions (Test Your Logic)

1. Which SOLID principle states a class should only have one reason to change?
2. Write a private constructor for a class named `DatabaseManager` to implement the Singleton pattern.
3. Why is Field Injection (using `@Autowired` on a variable) discouraged in Clean Architecture?
4. Write a JUnit 5 `@Test` method that asserts `10 / 2` equals `5`.
5. Use Mockito syntax to mock a `userRepo.findById(1)` method to return a dummy User object.
6. What is the difference between `log.debug()` and `log.error()`?
7. What are the three parts of a JSON Web Token (JWT)?
8. Why is the JWT Payload encoded but not encrypted?
9. Where are Java Objects stored: Stack Memory or Heap Memory?
10. What is the purpose of the Garbage Collector?
11. Write the Dockerfile command to use `openjdk:17-jdk-slim` as the base image.
12. What does "CI/CD" stand for?
13. Why should you never hardcode a database password into your `application.properties` before pushing to GitHub?
14. Explain the difference between a Unit Test and an Integration Test.
15. Why do we map Entities to DTOs before returning them in a `@RestController`?
16. What design pattern is used by Spring to inject dependencies?
17. Write a `@ControllerAdvice` class stub to handle `Exception.class`.
18. How does Swagger/OpenAPI help Frontend developers?
19. What is the purpose of the `assertThrows` method in JUnit?
20. Explain the Factory Design Pattern in one sentence.

IT STUDY HUB

B. University Viva Questions & Answers

Q: What are the SOLID principles?

Ans: A set of five design principles (Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) intended to make software designs more understandable, flexible, and maintainable.

Q: What is the Singleton Pattern?

Ans: A creational design pattern that restricts the instantiation of a class to one single instance across the entire application and provides a global point of access to it.

Q: What is the Factory Pattern?

Ans: A creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses or logic to alter the type of objects that will be created, hiding the 'new' keyword from the client.

Q: What is the difference between Unit Testing and Integration Testing?

Ans: Unit testing isolates a single method/class using mocks to verify its internal logic rapidly. Integration testing verifies that multiple components (like a Service, Database, and Network) work correctly together.

Q: What is Mockito used for?

Ans: Mockito is a mocking framework used in Unit Testing. It creates fake, programmable dummy objects (mocks) for external dependencies (like a database repository) so a class can be tested in complete isolation.

Q: What is SLF4J?

Ans: Simple Logging Facade for Java. It is an abstraction layer for various logging frameworks (like Logback or Log4j). It allows you to write log.info() and easily swap the underlying logging engine without changing your code.

Q: Why is logging preferred over System.out.println?

Ans: Logging provides severity levels (INFO, DEBUG, ERROR), can be easily routed to rolling files, includes timestamps and thread names automatically, and can be disabled in production without removing the code.

Q: What is a JWT?

Ans: JSON Web Token. A compact, URL-safe means of representing claims securely between two parties. It is heavily used for stateless authentication in REST APIs.

IT STUDY HUB

Q: What is the difference between Authentication and Authorization?

Ans: Authentication verifies WHO you are (e.g., username/password). Authorization verifies WHAT YOU CAN DO (e.g., checking if you have the 'ADMIN' role to delete a user).

Q: Why is a JWT considered 'Stateless'?

Ans: Because all the information necessary to authenticate the user (like user ID and roles) is cryptographically signed and stored entirely within the token itself. The server does not need to store session data in its RAM.

Q: What is JVM Heap Memory?

Ans: The runtime data area from which memory for all class instances (Objects) and Arrays is allocated. It is globally shared across all threads and managed by the Garbage Collector.

Q: What is JVM Stack Memory?

Ans: Memory allocated per thread that stores local primitive variables and method call execution frames. It is very fast and automatically clears when a method returns.

Q: What does the Garbage Collector do?

Ans: It automatically identifies and deletes Objects in Heap memory that are no longer reachable (have no active references) to free up RAM and prevent OutOfMemory errors.

Q: What is Docker?

Ans: A platform that uses OS-level virtualization to deliver software in packages called containers. Containers bundle the application code, runtime, system tools, and libraries together so it runs uniformly on any environment.

Q: What is a Dockerfile?

Ans: A text document containing all the commands a user could call on the command line to assemble a Docker image (e.g., FROM, COPY, RUN, ENTRYPOINT).

Q: What is CI/CD?

Ans: Continuous Integration and Continuous Deployment. It is an automated pipeline that compiles code, runs tests, builds containers, and deploys applications to live servers automatically when code is pushed to a repository.

Q: Why do we use DTOs (Data Transfer Objects)?

Ans: To prevent exposing internal database Entity structures to the web client, ensuring sensitive data (like passwords) is hidden, and decoupling the database schema from the API JSON contracts.

IT STUDY HUB

Q: What is Swagger / OpenAPI?

Ans: A suite of tools built around the OpenAPI Specification that automatically generates interactive, readable documentation for REST APIs based on Spring Controller annotations.

Q: What is Clean Architecture?

Ans: A software design philosophy that separates concerns into concentric layers, ensuring the core business logic does not depend on external frameworks, UI, or databases (Dependency Inversion).

Q: What is the purpose of Git and GitHub?

Ans: Git is a version control system used to track code changes and collaborate locally. GitHub is a cloud-based hosting service for Git repositories, enabling team collaboration, code reviews, and CI/CD pipelines.

C. Intermediate Interview Questions

Interview Q: How does the Single Responsibility Principle apply to the 3-Tier Architecture in Spring?

Ans: In Spring, a Controller is solely responsible for HTTP routing and JSON parsing. A Service is solely responsible for business logic. A Repository is solely responsible for database interaction. This perfectly adheres to the SRP, ensuring a change in database technology doesn't affect the HTTP layer.

Interview Q: How would you test a Service method that saves a User, without touching a real database?

Ans: I would use JUnit 5 and Mockito. I would annotate the Service with @InjectMocks and the UserRepository with @Mock. In the @Test method, I would call service.saveUser(user), and then use Mockito.verify(userRepository, times(1)).save(user) to assert that the Service correctly interacted with the Repository.

Interview Q: If a JWT is intercepted by a hacker, can they read the data inside it? How do you secure it?

Ans: Yes, the payload of a standard JWT is only Base64 encoded, not encrypted, meaning anyone can read it. Therefore, sensitive data (passwords, SSNs) must NEVER be put in the payload. Security is guaranteed via the JWT Signature, which prevents the hacker from altering the payload (e.g., changing their role to ADMIN) without invalidating the token.

IT STUDY HUB

Interview Q: Explain a scenario where a Memory Leak could occur in Java despite having a Garbage Collector.

Ans: A memory leak in Java occurs when objects are no longer needed by the application but are still accidentally referenced by a persistent root object. For example, adding temporary user session objects to a static HashMap and forgetting to remove them when the user logs out. The GC sees the HashMap holding the reference and refuses to delete the objects, eventually causing an OutOfMemoryError.

Interview Q: What is the difference between 'Constructor Injection' and 'Field Injection' in Spring?

Ans: Field injection uses @Autowired directly on private fields. It is heavily discouraged because it masks dependencies, prevents the use of 'final' fields (breaking immutability), and makes unit testing impossible without starting the Spring container. Constructor injection forces dependencies to be provided at instantiation, guaranteeing object state and making mocking in unit tests trivial.

Interview Q: How does Docker solve the 'It works on my machine' problem?

Ans: Traditional deployment relies on the host OS having the correct Java version, environment variables, and system libraries. Docker encapsulates the application, the specific JDK, and a minimal OS layer into a single, immutable Container Image. If the container runs on the developer's laptop, it is mathematically guaranteed to run identically on the AWS production server.

Interview Q: What is the role of GitHub Actions or Jenkins in a software project?

Ans: They serve as CI/CD automation servers. When a developer pushes a branch to GitHub, these tools intercept the code, spin up an isolated runner, compile the Java code, execute all JUnit tests, build the Docker image, and push it to a cloud registry. This enforces strict quality control, ensuring broken code never reaches production.

Interview Q: What is the Observer Design Pattern, and where might it be used?

Ans: It defines a one-to-many dependency so that when one object changes state, all its dependents (observers) are notified automatically. In enterprise Java, it is heavily used in Event-Driven architectures (like Spring ApplicationEvents) where a 'UserRegisteredEvent' is published, and independent EmailService and AnalyticsService observers react to it.

Interview Q: How do you prevent a 'Man-in-the-Middle' attack when transmitting JWTs?

Ans: A JWT must always be transmitted over HTTPS (SSL/TLS encrypted tunnels). If sent over plain HTTP, a hacker can easily sniff the network packet, steal the valid JWT, and perform a Replay Attack to impersonate the user.

IT STUDY HUB

Interview Q: Why is global exception handling (@ControllerAdvice) crucial for API security?

Ans: If an unhandled exception occurs (e.g., a SQL syntax error), standard Spring Boot returns a massive Stack Trace to the client. This stack trace reveals internal database structures, library versions, and class names—a goldmine for hackers. @ControllerAdvice intercepts the crash, hides the stack trace, logs it securely to the server files, and returns a generic '500 Internal Server Error' JSON to the client.

IT STUDY HUB

14. Module Summary and Key Terms

Key Terms Glossary

- **SOLID:** Five design principles for writing maintainable object-oriented code.
- **Design Pattern:** A proven, reusable solution to a commonly occurring software design problem.
- **Mocking:** Creating fake objects in Unit Tests to isolate the specific logic being tested.
- **JWT:** JSON Web Token. A secure, stateless standard for authenticating web requests.
- **Docker Container:** A lightweight, standalone, executable package containing everything needed to run an app.
- **CI/CD:** Continuous Integration / Continuous Deployment. Automating testing and server deployment.

Module Summary

In this final module, you transitioned from writing code that simply "functions" to architecting **Production-Ready Software**. You learned how to structure applications using the **SOLID principles** and **Design Patterns** to ensure long-term maintainability.

You secured your application against crashes using **Test-Driven Development (TDD)** with **JUnit** and **Mockito**. You prepared for the real world by adding **SLF4J Logging** for server monitoring, **Swagger** for API documentation, and **JWT** for robust security. You peeked under the hood of the JVM to understand **Garbage Collection**.

Finally, you learned the critical **DevOps** pipeline: containerizing your Java application with **Docker**, pushing it via **Git**, and automating its delivery through a **CI/CD pipeline**.

15. The Java Developer Roadmap

Now that you have completed this series, here are your next steps to secure a Junior Backend Developer role:

1. **Build a Portfolio:** Create 2-3 complete Spring Boot REST APIs (e.g., E-commerce backend, Blog platform). Put the code on your GitHub.
2. **Learn SQL Deeply:** Go beyond CRUD. Learn Joins, Group By, Indexes, and optimization.
3. **Master Git:** Understand branching, merging, pull requests, and resolving merge conflicts.
4. **Explore the Cloud:** Learn how to deploy your Docker containers to AWS (EC2, Elastic Beanstalk) or Heroku.

IT STUDY HUB

5. **Learn Frontend Basics:** Understand how React or Angular consumes the JSON your Java backend provides.

CONGRATULATIONS!

YOU HAVE COMPLETED THE COMPLETE JAVA PROGRAMMING SERIES

Look how far you've come:

Modules 1–6 → Java Fundamentals (Syntax, OOP, Collections, File I/O)
Modules 7–9 → Advanced Java (Generics, Lambdas, Streams, Multithreading)
Modules 10–11 → Database & Backend (SQL, JDBC, Spring Boot, JPA, REST APIs)
Module 12 → Production & Professional Dev (Architecture, Testing, JWT, Docker, CI/CD)

You now possess the foundational knowledge of a professional Enterprise Software Engineer. Keep coding, keep building, and never stop learning.

IT STUDY HUB SERIES